

Лабораторная работа № 3

Модель открытого текста

Вариант №15

Ф. И. О. студента: Рау Алексей Евгеньевич

Группа: ФИТ-231

Проверил: Белим С.Ю.

Дата: 11.11.2025

Основные сведения

Энтропия открытого текста определяется формулой:

$$H_k(T) = - \sum_{i=1}^n p(z_i^k) \log_2(p(z_i^k))$$

Результаты

Энтропия для k-грамм открытого текста:

$$k = 1, H_1 = 4.4515, \frac{H_1}{1} = 4.4515$$

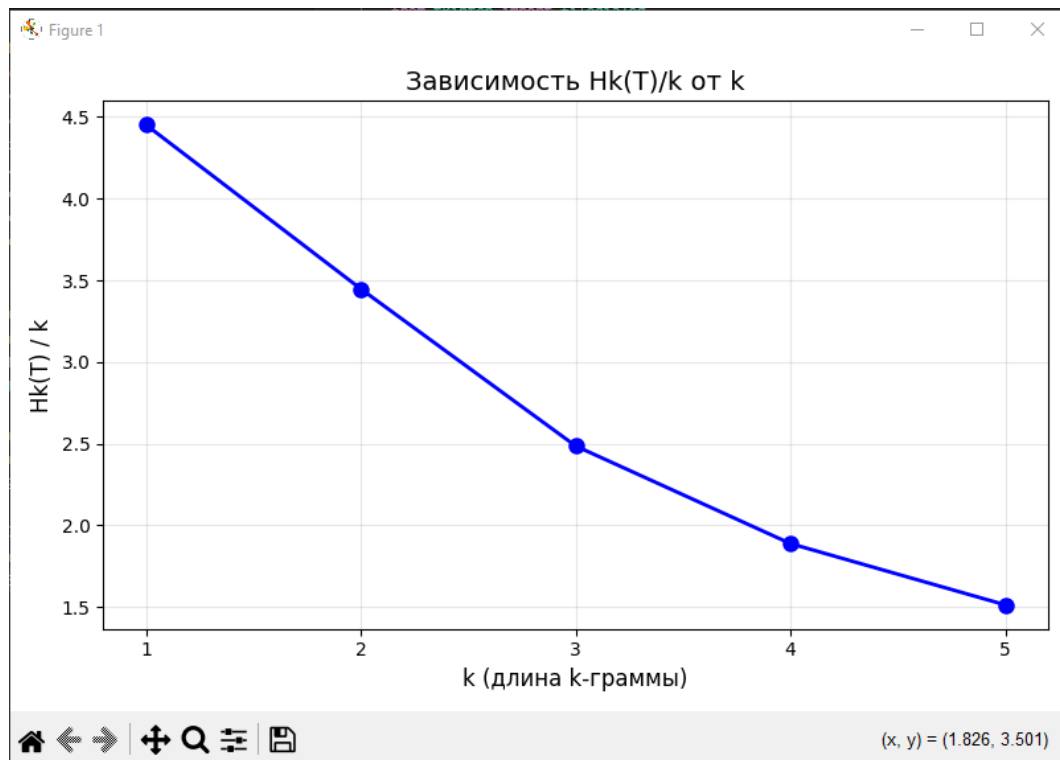
$$k = 2, H_2 = 6.8943, \frac{H_2}{2} = 3.4471$$

$$k = 3, H_3 = 7.4577, \frac{H_3}{3} = 2.4859$$

$$k = 4, H_4 = 7.5537, \frac{H_4}{4} = 1.8884$$

$$k = 5, H_5 = 7.5670, \frac{H_5}{5} = 1.5134$$

График зависимости H_k/k от k .



Асимптотическое значение H_k/k при $k \rightarrow \infty$: $\frac{H_k}{k} = 0$

Код программы

```
import tkinter as tk
from tkinter import filedialog
import math
import matplotlib.pyplot as plt
import sys
from io import StringIO

def get_text_input():
    print("\n" + "=" * 50)
    print("Выберите способ ввода текста:")
    print("1. Ввод с консоли")
    print("2. Чтение из файла")

    choice = input("\nВаш выбор (1-2): ").strip()

    if choice == '1':
        print("\n" + "=" * 50)
        return input("Введите текст: ")
    elif choice == '2':
        return read_from_file_dialog()
    else:
        print("Неверный выбор!")
        return None

def read_from_file_dialog():
    try:
        root = tk.Tk()
        root.withdraw()
        root.attributes('-topmost', True)

        file_path = filedialog.askopenfilename(
            title="Выберите текстовый файл",
```

```

        filetypes=[("Текстовые файлы", "*.txt"), ("Все файлы", "*.*")]
    )

    root.destroy()

    if not file_path:
        print("Выбор файла отменен.")
        return None

    print(f"Выбран файл: {file_path}")

    with open(file_path, 'r', encoding='utf-8') as f:
        content = f.read().strip()
        print(f"Файл успешно прочитан. Размер: {len(content)} символов")
        return content

    except Exception as e:
        print(f"Ошибка при чтении файла: {e}")
        return None

def clean_text(text):
    russian_letters = "абвгдеёжзийклмнопрстуфхцчшщъыьэюя"
    cleaned = []
    for ch in text.lower():
        if ch in russian_letters:
            cleaned.append(ch)
    return ''.join(cleaned)

def get_kgrams(text, k):
    kgrams = {}
    total = 0
    for i in range(len(text) - k + 1):
        kgram = text[i:i+k]
        kgrams[kgram] = kgrams.get(kgram, 0) + 1
        total += 1
    for key in kgrams:
        kgrams[key] = kgrams[key] / total
    return kgrams

def calculate_entropy(kgrams):
    entropy = 0.0
    for freq in kgrams.values():
        if freq > 0:
            entropy -= freq * math.log2(freq)
    return entropy

def analyze_text(text):
    cleaned = clean_text(text)
    if len(cleaned) < 5:
        print("Текст слишком короткий после очистки!")
        return None

    print(f"Текст после очистки: {len(cleaned)} символов")
    if len(cleaned) < 100:
        print(f"Первые 100 символов: {cleaned[:100]}...")

    results = []
    for k in range(1, 6):
        kgrams = get_kgrams(cleaned, k)
        Hk = calculate_entropy(kgrams)
        Hk_per_k = Hk / k
        results.append({

```

```

        'k': k,
        'Hk': Hk,
        'Hk_per_k': Hk_per_k,
        'unique_kgrams': len(kgrams)
    })
    print(f"k={k}: Hk = {Hk:.4f}, Hk/k = {Hk_per_k:.4f}, уникальных k-грамм: {len(kgrams)}")

    return results, cleaned

def plot_results(results):
    k_values = [r['k'] for r in results]
    Hk_per_k_values = [r['Hk_per_k'] for r in results]

    plt.figure(figsize=(8, 5))
    plt.plot(k_values, Hk_per_k_values, 'bo-', linewidth=2, markersize=8)
    plt.xlabel('k (длина k-граммы)', fontsize=12)
    plt.ylabel('Hk(T) / k', fontsize=12)
    plt.title('Зависимость Hk(T)/k от k', fontsize=14)
    plt.grid(True, alpha=0.3)
    plt.xticks(k_values)
    plt.tight_layout()

    print("Закройте окно с графиком, чтобы продолжить работу программы.")
    plt.show()

def save_results_to_file(console_output, filename="results.txt"):
    try:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(console_output)

        print(f"Результаты успешно сохранены в файл: {filename}")
        return True

    except Exception as e:
        print(f"Ошибка при сохранении в файл: {e}")
        return False

def main():
    print("=" * 60)
    print("ЛАБОРАТОРНАЯ РАБОТА №3: МОДЕЛЬ ОТКРЫТОГО ТЕКСТА")
    print("=" * 60)

    while True:
        text = get_text_input()

        if text is None:
            print("Текст не получен. Попробуйте еще раз.")
            continue

        if len(text.strip()) == 0:
            print("Текст пустой. Попробуйте еще раз.")
            continue

        old_stdout = sys.stdout
        console_output = StringIO()
        sys.stdout = console_output

        analysis_result = analyze_text(text)
        if analysis_result is None:
            sys.stdout = old_stdout
            continue

```

```

results, cleaned_text = analysis_result

print("\n" + "=" * 60)
print("РЕЗУЛЬТАТЫ АНАЛИЗА")
print("=" * 60)
print(f"Длина очищенного текста: {len(cleaned_text)} символов")
print("\nТаблица результатов:")
print("-" * 60)
print("k\tHk(T)\t\tHk(T)/k\t\tУникальных k-грамм")
print("-" * 60)

for r in results:

print(f"{r['k']}\t\t{r['Hk']:.4f}\t\t{r['Hk_per_k']:.4f}\t\t{r['unique_kgrams']}")

sys.stdout = old_stdout

full_output = console_output.getvalue()

print(full_output)

try:
    plot_results(results)
except Exception as e:
    print(f"Ошибка при построении графика: {e}")

print("\n" + "=" * 60)
save_choice = input("Сохранить результаты в файл? (y/n):
").strip().lower()

if save_choice == 'y':
    filename = input("Введите имя файла (без расширения, по умолчанию
'results'): ").strip()
    if not filename:
        filename = "results"
    if not filename.endswith('.txt'):
        filename += '.txt'

    save_results_to_file(full_output, filename)

print("\n" + "=" * 60)
choice = input("Хотите проанализировать другой текст? (y/n):
").strip().lower()
if choice != 'y':
    break

print("\nПрограмма завершена.")

if __name__ == "__main__":
    main()

```